BLOCKCHAIN TECHNOLOGY PRACTICAL NOTES

Answer-sheet-ready notes for the five assignments shown in your practical sheet

Suggested answer-sheet format: Aim → Notes/Theory → Important formula or terms → Procedure/working (if asked) → Observation/Output → Conclusion.

Important: The practical sheet specifies the tasks, but it does not specify exact wallet settings, Solidity code, testnet, deployment tool or measured gas values. These notes explain the concepts; write exact values and implementation details from your own practical.

Experiment 1 – Installation of MetaMask and Study of Ether Spent per Transaction

Aim: To install and configure MetaMask and study the amount of Ether spent during a blockchain transaction.

Notes / Theory

1. **MetaMask** is a cryptocurrency wallet that allows users to manage Ethereum-compatible accounts and interact with decentralised applications (DApps).

2. MetaMask is commonly available as a browser extension and a mobile application. It can be used to create a new wallet or import an existing wallet.

3. A wallet contains an account address and cryptographic keys. The public address can be shared to receive assets, while the private key and Secret Recovery Phrase must be kept confidential.

4. **Ether (ETH)** is the native cryptocurrency of the Ethereum network. It is used to pay transaction fees and can also be transferred between accounts.

5. When a user sends a transaction, MetaMask displays the transaction details, including the destination, amount and estimated network fee, before confirmation.

6. **Gas** is the unit used to measure the computational work required to execute an operation on Ethereum. More complex operations generally require more gas.

7. The transaction fee is calculated using the amount of gas used and the effective gas price. The fee is paid in ETH on Ethereum mainnet.

8. A simple ETH transfer usually consumes less gas than a smart-contract interaction because smart contracts require computation and may update blockchain storage.

9. After confirmation, the transaction is recorded on the blockchain. Its status and details can be checked using a blockchain explorer.

Important formulas / terms

- Transaction fee = **Gas used × Effective gas price**
- 1 ETH = $\mathbf{10^{18}}$ **Wei**
- Gas limit = maximum gas the sender is willing to allow for the transaction; it is not necessarily the gas actually used.

Precaution / Important point

The exact fee is not fixed. It varies with network conditions, transaction complexity and gas price. A displayed estimate can differ from the final fee.

Conclusion

MetaMask provides an interface to manage Ethereum accounts and approve transactions. The Ether spent on a transaction depends on gas usage and the effective gas price.

Experiment 2 – Create a Wallet Using MetaMask for Cryptocurrency Transactions

Aim: To create a cryptocurrency wallet using MetaMask and understand how it is used for blockchain transactions.

Notes / Theory

1. A **cryptocurrency wallet** is a software or hardware tool that manages the keys used to access and control blockchain assets. The assets themselves remain recorded on the blockchain.

2. MetaMask is a non-custodial wallet. This means the user is responsible for protecting the wallet's recovery phrase and private keys.

3. To create a wallet, install the official MetaMask extension or application and follow its wallet-creation instructions.

4. During setup, MetaMask provides a **Secret Recovery Phrase**. It is used to restore access to the wallet and must be stored securely and privately.

5. A wallet address is a public identifier that can be shared with others to receive cryptocurrency. It is different from the private key.

6. To receive cryptocurrency, share the correct public address and make sure the sender uses the correct blockchain network.

7. To send cryptocurrency, enter the recipient's address and amount, review the network and fee details, and confirm only if the details are correct.

8. Every transaction must be signed using the wallet's private key. The private key proves control of the account without being revealed to the network.

9. Transactions are broadcast to the blockchain network, checked by network participants and included in a block when accepted.

10. Never share the Secret Recovery Phrase or private key. A person who obtains them may gain control of the wallet and its funds.

Important formulas / terms

- Public address: used to receive assets and identify an account.
- Private key: secret cryptographic key used to authorise/sign transactions.
- Secret Recovery Phrase: backup information used to restore wallet access.

Precaution / Important point

Use only the official MetaMask website or app store listing. Never enter a recovery phrase into an unknown website, message or form. Testnet tokens generally have no real monetary value.

Conclusion

MetaMask allows users to create and manage a non-custodial wallet, receive cryptocurrency and sign transactions while keeping control of their keys.

Experiment 3 – Bank Account Smart Contract on a Test Network

Aim: To write and deploy a smart contract on an Ethereum test network to perform deposit money, withdraw money and show balance operations.

Notes / Theory

1. A **smart contract** is a program stored and executed on a blockchain. It runs according to predefined rules when a transaction or call invokes its functions.

2. Solidity is a high-level programming language used to write smart contracts for the Ethereum Virtual Machine (EVM).

3. A bank-account smart contract can maintain a balance and provide functions for depositing, withdrawing and viewing the balance.

4. The **deposit** operation adds an allowed amount to the account or contract balance. In a payable Solidity function, Ether can be received by the contract.

5. The **withdraw** operation transfers an allowed amount out of the contract, after checking that sufficient balance is available.

6. The **show balance** operation reads the current balance. A read-only function can be declared using the **view** keyword when it does not modify contract state.

7. A **require()** statement is commonly used to check conditions, such as a positive deposit amount or sufficient balance before withdrawal.

8. Deploying a contract means sending its compiled bytecode to the blockchain. The deployment transaction is recorded and receives a contract address.

9. A test network (testnet) behaves like a blockchain environment for development and testing. Testnet tokens are used to pay test transaction fees and generally have no real monetary value.

10. State-changing operations such as deployment, deposit and withdrawal require transactions and gas. Reading public state through a view call normally does not require an on-chain transaction fee when called locally through an RPC provider.

Important formulas / terms

- Contract balance can be read in Solidity using **address(this).balance**.
- Ether units: **1 ether = 10^{18} wei**.
- Transaction fee: **Gas used × Effective gas price**.

Precaution / Important point

A demonstration bank contract is not a real banking system. A production contract needs careful access control, security review and safe withdrawal logic. The exact code and access rules depend on the program written for the practical.

Conclusion

A Solidity smart contract can implement deposit, withdrawal and balance operations on an Ethereum test network. Gas is consumed by deployment and state-changing transactions.

Experiment 4 – Student Data in Solidity Using Structures, Arrays and Fallback

Aim: To write a Solidity program to create student data using structures, arrays and a fallback function, deploy it on Ethereum, and observe transaction fee and gas values.

Notes / Theory

1. **Solidity** is a contract-oriented programming language used to develop smart contracts for Ethereum and other EVM-compatible blockchains.

2. A **structure (struct)** is a user-defined data type that groups related variables of different data types under one name. For example, a Student structure may contain an ID, name and marks.

3. An **array** stores multiple values of the same declared element type. In Solidity, arrays can be fixed-size or dynamic-size.

4. A dynamic array can grow or shrink during contract execution using operations such as **push()** and **pop()** when applicable.

5. A **fallback function** is a special function that is executed when a contract receives a call that does not match any function, or in certain cases when it receives Ether without data. It is declared using the **fallback** keyword.

6. A fallback function must be external. It can be marked payable if it is intended to receive Ether.

7. Contract deployment stores the compiled contract bytecode on the blockchain and creates a contract address.

8. Adding or changing student records modifies blockchain state and requires a transaction. Such transactions consume gas.

9. Gas measures the computational work performed by the EVM. Gas used depends on the operations executed and whether storage is read or modified.

10. The transaction fee depends on gas used and the effective gas price. The gas limit is the maximum amount of gas permitted for the transaction.

11. After deployment and execution, transaction details can be inspected in the development environment or a blockchain explorer, including gas used and transaction fee.

Important formulas / terms

- Transaction fee = **Gas used × Effective gas price**
- 1 ether = **10^{18} wei**
- Struct example (concept): **struct Student { uint id; string name; }**

Precaution / Important point

A fallback function is not automatically called for every ordinary function call; it is used when no function selector matches, or under the relevant empty-calldata conditions. The exact gas and fee values depend on the deployed code and network conditions.

Conclusion

Solidity structures and arrays can organise student records, while a fallback function handles unmatched calls or applicable Ether transfers. Deployment and state changes consume gas.

Experiment 5 – Survey Report on Types of Blockchains and Their Real-Time Use Cases

Aim: To study different types of blockchains and their real-world applications.

Notes / Theory

1. A **blockchain** is a distributed digital ledger in which transactions are grouped into blocks and linked using cryptographic hashes.

2. Copies of the ledger are maintained by multiple participating computers (nodes). The network uses a consensus mechanism to agree on the valid state of the blockchain.

3. Once confirmed and added to the chain, records are designed to be difficult to alter without detection. This supports transparency and tamper resistance.

4. **Public blockchain:** An open network in which anyone can generally read data, submit transactions and participate according to the network rules. Examples include Bitcoin and Ethereum.

5. **Private blockchain:** A blockchain controlled by one organisation, which decides who can join and perform permitted activities.

6. **Consortium blockchain:** A permissioned blockchain governed by a group of organisations rather than a single organisation.

7. **Hybrid blockchain:** Combines elements of public and private blockchains, allowing selected information or operations to be public while keeping other data restricted.

8. **Finance and payments:** Blockchain can support cryptocurrency transfers, settlement and shared transaction records.

9. **Supply chain:** Participants can record the movement and status of goods to improve traceability and coordination.

10. **Healthcare:** Blockchain-based systems may support controlled sharing and audit trails for health records, subject to privacy and regulatory requirements.

11. **Digital identity:** Blockchain can support verifiable credentials and identity-related records.

12. **Smart contracts:** Programs on a blockchain can automate agreed rules, such as releasing payment after specified conditions are met.

Important formulas / terms

- Block (conceptually) contains transaction data, a timestamp and a cryptographic reference/hash to the previous block.
- Hashing converts input data into a fixed-length digest; changing the input produces a different digest with overwhelming probability.

Precaution / Important point

Blockchain is not suitable for every application. Scalability, transaction cost, privacy, regulation and governance must be considered. The use cases above are general examples, not claims about a particular deployed system.

Conclusion

Blockchains can be public, private, consortium or hybrid. Their shared-ledger and smart-contract features can support applications in finance, supply chains, healthcare and digital identity.